

14: Handling Exceptions and Writing better code

Exception Handling with Try-Catch Block:

- It's essential to wrap the code in a try-catch block to handle exceptions effectively.
- Multiple catch blocks can be used for a single try block, where each catch handles a specific exception.
- Include a generic catch block to handle unexpected exceptions.

Using the Finally Block:

- If an exception occurs, any code after the exception won't be executed until the catch block.
- The finally block executes after the try-catch block, regardless of whether an exception occurs.
- Use the finally block to close resources like database connections, statements, and result sets.

Declaring Common Interfaces at the Top:

- For better readability and organization, declare common interfaces like:

Connection connect = null;

Statement statement = null;

Common Steps for All Operations:

- Steps like creating connections, loading drivers, and closing resources are common across different JDBC operations.
- These steps can be encapsulated in a **utility class** to reduce redundancy and promote reusability.

Using Static Block for Driver Loading:

- Load the database drivers in a static block, so they get registered when the class is loaded.
- The static block is executed during class loading.

Creating Static Method for Connection:

- Use a static method for creating the connection, allowing direct access via the class name.

Closing Resources with a Utility Method:

- Write a separate method to close the connection and other resources.

Reusability through Utility Class:

- The utility class can be called from other classes using the class name directly, improving code maintainability.

Utility Class:

```
import java.sql.*;

public class jdbcConnection{
    static {
        // Load and Register the Driver
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }

    public static Connection getJdbcConnection() throws SQLException{
        //resources used in jdbc application
        Connection connection=null;
```

```

        //2. Establish the connection
        String url= "jdbc:mysql://localhost:3306/db_name";
        String user= "root";
        String password= "root";

        connection=DriverManager.getConnection(url, user, password);
        return connection;
    }

    public static void closeConnection(Statement statement, Connection
connection) throws SQLException{
        statement.close();
        connection.close();
    }
}

```

Insertion Code:

```

import java.sql.*;

public class InsertionClass {

    public static void main(String[] args) {

        Connection connect = null;

        Statement statement = null;
    }
}

```

```
try {  
    connect = jdbcConnection.getJdbcConnection();  
    //creating statement  
    statement = connect.createStatement();  
  
    //execute query  
    String sql = "INSERT INTO studentinfo(id, sname, sage, scity) VALUES  
(3, 'Rohit', 17, 'Pune')";  
  
    int rowAffected = statement.executeUpdate(sql);  
    //process the result  
    if (rowAffected == 0) {  
        System.out.println("Unable to insert the data");  
    } else {  
        System.out.println("Data Inserted Successfully!");  
    }  
}  
catch (SQLException e) {  
    e.printStackTrace();  
}  
catch (Exception e) {  
    e.printStackTrace();  
}
```

```
} finally {  
    //close the resources  
    try {  
        jdbcConnection.closeConnection(statement, connect);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}  
  
}
```